Catch game

1. Game Introduction

- Start Docker

After entering the main board desktop, open the terminal and run the following command to start the Dofbot corresponding container:

```
./Docker_Ros.sh
```

Then enter the following command in the docker terminal to start jupyter-lab,

```
cd
jupyter-lab --allow-root
```

Then enter the main board's IP address plus :9999 in the browser,

```
IP:9999         // Example: 192.168.1.11:9999
```

You place, I grab game path: /root/dofbot_ws/src/dofbot_color_sorting/你放我抓.ipynb

Start the code block, the robotic arm automatically reaches the recognition position, place the blocks in the gray grid, after the robotic arm continuously recognizes correctly 10 times, it automatically grabs and places them in the corresponding color grid.

- Import header files

```
#!/usr/bin/env python
# coding: utf-8
import cv2 as cv
import threading
from time import sleep
from dofbot_config import *
import ipywidgets as widgets
from IPython.display import display
from color_sorting import color_sorting
```

- Initialize robotic arm position

```
import Arm_Lib
Arm = Arm_Lib.Arm_Device()
joints_0 = [90, 135, 0, 45, 90, 30]
Arm.Arm_serial_servo_write6_array(joints_0, 1000)
```

- Create instance, initialize parameters

```
# Create instance
sorting = color_sorting()
# Initialize mode
model = 'General'
```

```
# Color HSV threshold
color_hsv = {"red" : ((0, 43, 46), (10, 255, 255)),
        "green" : ((35, 43, 46), (77, 255, 255)),
        "blue" : ((100, 43, 46), (124, 255, 255)),
        "yellow": ((26, 43, 46), (34, 255, 255))}
# HSV parameter path
HSV_path="/root/dofbot_ws/src/dofbot_color_sorting/HSV_config.txt"
# Read HSV configuration file, update HSV values
try: read_HSV(HSV_path,color_hsv)
except Exception: print("Read HSV_config Error!!!")
```

- Create controls

```
# Create control layout
button_layout = widgets.Layout(width='200px', height='70px',align_self='center')
# Output print
output = widgets.Output()
# Exit button
exit_button=widgets.Button(description='Exit',button_style='danger',layout=butto
n_layout)
# Image control
imgbox = widgets.Image(format='jpg', height=480, width=640,
layout=widgets.Layout(align_self='center'))
# Vertical placement
controls_box = widgets.VBox([imgbox,
exit_button],layout=widgets.Layout(align_self='center'))
# ['auto','flex-start', 'flex-end', 'center', 'baseline', 'stretch', 'inherit',
'initial', 'unset']
```

- Control button

```
def   exit_button_Callback(value):
    global model
    model = 'Exit'
#    with output: print(model)
exit_button.on_click(exit_button_Callback)
```

- Main program

```
def camera():
    # Open camera
    capture = cv.VideoCapture(0)
    # Loop execution when camera is normally opened
    while capture.isOpened():
        try:
            # Read each frame of the camera
            _, img = capture.read()
            # Unify image size
            img = cv.resize(img, (640, 480))
            # Get movement information
            img = sorting.Sorting_grap(img,   color_hsv)
            if model == 'Exit':
                cv.destroyAllWindows()
                capture.release()
                break
            # Add text
```

```
            imgbox.value = cv.imencode('.jpg', img)[1].tobytes()
        except KeyboardInterrupt:capture.release()
```

- Start

```
display(controls_box,output)
threading.Thread(target=camera, ).start()
```

- Example image

For detailed game code, see /root/Dofbot_2D/src/dofbot_color_sorting/你放我抓.ipynb

2. Library Code Design

- HSV filter target color

```
# Convert image to HSV.
HSV_img = cv.cvtColor(mask, cv.COLOR_BGR2HSV)
# Filter elements located between two arrays.
img = cv.inRange(HSV_img, lowerb, upperb)
# Set non-mask detection part to all black
mask[img == 0] = [0, 0,   0]
```

- Morphological transformation

```
# Get structural elements of different shapes
kernel = cv.getStructuringElement(cv.MORPH_RECT, (5, 5))
# Morphological closing operation
dst_img = cv.morphologyEx(mask, cv.MORPH_CLOSE, kernel)
```

- Find contours in binary image

```
# Convert image to grayscale
dst_img = cv.cvtColor(dst_img, cv.COLOR_RGB2GRAY)
# Image binarization operation
ret, binary = cv.threshold(dst_img, 10, 255, cv.THRESH_BINARY)
# Get contour point set (coordinates)
contours, heriachy = cv.findContours(binary, cv.RETR_EXTERNAL,
cv.CHAIN_APPROX_SIMPLE)
```

- Calculate contour bounding box

```
for i, cnt in enumerate(contours):
    # boundingRect function calculates bounding box values, x, y are coordinate
values, w, h are the width and height of the rectangle
    x, y, w, h = cv.boundingRect(cnt)
    # Calculate the area of the contour
    area = cv.contourArea(cnt)
```

Obtain target, drive robotic arm to grab, grabbing process is as follows:

```
    Lift robotic arm -> Release gripper -> Move to block position -> Tighten
gripper -> Lift up -> Move to target position -> Release gripper -> Robotic arm
reset.
```